Bridging the Ontology Gap

Making Domain Knowledge Accessible with OSLC and AI

A platform for defining, instantiating, and activating standard domain models

The Paradox

The people who **know the most** about a domain — municipal service managers, engineers, clinicians — are the **least equipped** to capture that knowledge in formal ontologies.

The people who **can build ontologies** don't have the domain expertise.

The Result

- Valuable reference models get created once with great effort, then **stagnate**
- Or they never get created at all
- Even when they exist, most stakeholders **can't access or use them**
- We fall back to collecting documents and addressing problems of the moment

> *"We know what we know, but we can't put it into a form that the organization can systematically use."*

What If...

- What if **describing** your domain vocabulary and constraints was enough to get a **working tool**?
- What if **AI** could read your existing documents and **populate the model**?
- What if **anyone** could browse, query, and contribute — without knowing RDF or SPARQL?

oslc4js makes this real.

MRM (MISA Municipal Reference Model) is our running example.

The Semantic Value Chain

Define — Instantiate — Activate

Three Layers of Shared Meaning

To make domain knowledge actionable across an enterprise, three layers must work together:

Layer	Question it answers	Character
1. Define	What kinds of things exist? What properties and relationships do they have?	Schema / vocabulary governance
2. Instantiate	What are the actual artifacts — requirements, services, processes — their content, links, and governance state?	Instance creation and management
3. Activate	What decisions, compliance evidence, analyses, and actions can we derive from the governed data?	Value delivery and outcomes

This maps onto the classic *schema / instance / use* distinction from information architecture — applied to the OSLC linked data ecosystem.

Where Deployments Fail

Most OSLC deployments invest heavily in Layer 2 (tools, adapters, data migration) without adequate investment in the other layers:

- **Without Layer 1** (Define) — Layer 2 produces a connected but *semantically incoherent* graph. Links exist but mean different things in different tools.
- **Without Layer 2 governance** (versioning, configuration management) — Layer 3 can't answer versioned questions. All you get is a snapshot of today's state.
- **Without Layer 3** (Activate) — Layers 1 and 2 produce a beautifully governed but *unused* knowledge graph. The classic ontology project failure mode.

> *The three barriers that follow are concrete manifestations of gaps in these layers.*

How oslc4js Addresses Each Layer

Layer 1 — Define: Declarative vocabularies and OSLC ResourceShapes formalize domain knowledge as REST API contracts. Vocabulary governance tools (e.g., TopBraid EDG) manage the ontology lifecycle.

Layer 2 — Instantiate: The OSLC server hosts governed instances. AI via MCP acts as a first-class participant — creating, linking, and validating resources directly. Configuration management (GCM) adds the temporal dimension.

Layer 3 — Activate: Three mechanisms deliver value:

- *Analytical* — SPARQL/LQE queries for traceability, compliance, coverage
- *Agentic* — MCP endpoint lets AI reason over live data and propose actions
- *Operational* — Tool integrations surface linked data inline in native environments

Barrier 1

"Creating ontology-based models is too hard"

The Traditional Approach

To turn a domain ontology into a usable tool, you need to:

1. Hand-author RDF/OWL vocabularies
2. Write SHACL or OSLC shape constraints
3. Build a custom CRUD application
4. Wire up storage and persistence
5. Build a user interface

This requires a **rare combination** of ontology expertise + full-stack development.

MRM: The Challenge

MISA defined **8 primary resource types** for municipal service management:

Program	Service	Process
Resource	Outcome	OrganizationUnit
TargetGroup	Need	

with rich relationships between them. But building a **tool** to manage them? That was a separate, expensive project.

The oslc4js Solution: Describe, Don't Build

You provide **three declarative artifacts**:

1. **Vocabulary** (Turtle) — your domain terms and relationships
   `mrm:Program`, `mrm:Service`, `mrm:administeredBy`...
2. **Resource Shapes** (Turtle) — constraints on each type
   Properties, cardinalities, value types, allowed values
3. **Catalog Template** (Turtle) — the services you want, e.g.,
   Creation factories, query capabilities, dialogs

oslc4js provides the rest.

What You Get Automatically

- **REST API** — OSLC 3.0 compliant CRUD + query
- **Query services** — A full implementation of OSLC with optional storage-service specific endponts such as SPARQL
- **Creation dialogs** — generated from resource shapes
- **Selection dialogs** — for selecting resource across tools for link creation
- **Compact preview** — for viewing resources across tools
- **Bulk import** — load existing RDF datasets
- **Pluggable storage** — Jena/Fuseki, filesystem, MongoDB — or implement the simple StorageService interface on your existing tools and repositories to adapt them for OSLC
- **oslc-browser** — column-based navigation, property views, link traversal, diagrams — a working UI from day one, with custom UX layered on later as needed

MRM: From Template to Running Server

A single declarative Turtle file defines all 8 OSLC managed resource types with their creation factories, dialogs, query capabilities, and resource shape references.

```
npm start
→ Fully functional MRM server at localhost:3002
```

Demo: Create a ServiceProvider for "City of Ottawa," and view Programs, Services, and Processes through the browser.

Value Delivered

- **Weeks** of custom development replaced by **declarative configuration**
- Existing tools can be OSLC-enabled by implementing a **simple storage a service dapter**
- Domain experts focus on **vocabulary and shapes** (what they know), not software (what they don't)
- Users get a **working browser UI from day one** — no UX project required to start getting value
- Any new domain follows the **same pattern** — the investment is reusable

Barrier 2

"Connecting domains requires manual linking expertise"

The Silo Problem

Real-world problems **span domains**:

A municipal service involves programs, budgets, IT systems, regulatory requirements, target populations.

Each domain may have its own vocabulary and tools — but the **value is in the connections** between them, the **Digital Thread**.

Traditionally, cross-domain linking requires someone who understands both the RDF mechanics and the semantic relationships — and does it **manually, link by link**.

Result: domains stay **siloed**, or links are **incomplete and stale**.

OSLC: Linked Data by Design

- Every resource has a **URI**, every relationship is a **navigable link**
- **Discovery protocol** — like a yellow pages for data:
  Catalog → ServiceProviders → Services → Factories/Queries → Shapes
- Any OSLC client can **discover** what's available without prior knowledge
- Resource shapes declare which properties are **links** to other types — the link structure is part of the model, not an afterthought
- Multiple OSLC servers on different domains can **cross-link by URI** — no shared database required
- Vocabularies and services can easily be extended by updating the catalog templates

MRM: Linked Municipal Services

A "Water Treatment" Service links to:

- The **Program** that administers it (`mrm:administeredBy`)
- The **Processes** that implement it (`mrm:processes`)
- The **OrganizationUnit** accountable for it (`mrm:accountableTo`)
- The **TargetGroups** it serves
- The **Outcomes** it produces
- The **Needs** it addresses

Demo: Navigate Service → Processes → Resources through link traversal in oslc-browser.

Value Delivered

- Cross-domain relationships are **first-class citizens**, not afterthoughts
- OSLC discovery means any new tool or domain can **find and link** to existing resources without custom integration
- The browser makes link traversal **intuitive** — explore by clicking, not by writing queries
- Domains can be developed **independently** and linked **incrementally** — you don't need everything in place on day one

Barrier 3

"Creating and consuming models requires specialized skills"

Two Sides of the Same Problem

Creating:
Populating an ontology means analyzing large volumes of unstructured material — policy documents, service catalogs, council reports — and translating them into structured, linked resources.

Tedious. Error-prone. Requires both domain knowledge *and* RDF skills.

Consuming:
Even when populated, most stakeholders can't access the model. Querying requires e.g., SPARQL. Analysis — gap analysis, cost/benefit, impact assessment — requires exporting data and building custom reports.

Two Access Paths

For humans: oslc-browser

Connect to any OSLC server, browse the catalog, navigate resources, view properties, traverse links, visualize diagrams. No RDF or SPARQL training required.

For AI: oslc-mcp-server

A generic Model Context Protocol (MCP) server — the emerging standard for connecting AI to external data. Discovers any OSLC server's capabilities at startup and exposes them as tools an LLM can call. The LLM reads vocabulary, shapes, and catalog through reflective MCP resources — it **learns the domain model on its own**.

AI as Creator: Documents → Models

The workflow that changes the population equation:

1. LLM reads MCP resources (`oslc://vocabulary` , `oslc://shapes`) to **learn the domain**
2. User provides a document — policy paper, service catalog, council minutes
3. LLM **identifies** domain entities and relationships in the text
4. LLM calls create tools (`create_program` , `create_service` , `create_process` ...)
5. LLM calls `update_resource` to add **cross-references** between resources
6. Results are immediately **browsable** in oslc-browser

The domain expert's role shifts from **data entry** to **review and validation**.

AI as Consumer: Models → Insight

The same MCP connection lets AI work *with* the populated data:

- **Questions:** *"Which services have no assigned process?"*
  LLM queries via MCP, reports in plain language
- **Gap analysis:** *"Compare our catalog against the reference model — what's missing?"*
  LLM reads both, identifies gaps
- **Cost/benefit:** *"Which programs deliver the most outcomes relative to resources consumed?"*
  LLM traverses links, synthesizes a summary
- **Impact analysis:** *"If we reorganize Parks, which services and processes are affected?"*
  LLM follows OrganizationUnit links to dependencies

MRM: Before and After

Before:

A consultant spends weeks reading municipal documents, manually creating RDF. The model is a static artifact. Getting answers requires a SPARQL specialist. Updates require the consultant again.

After:

A service manager points AI at a service delivery report. AI populates programs, services, processes. Manager reviews in oslc-browser, corrects a few links. Then asks: *"What target groups are underserved by our current programs?"* AI queries the model and delivers a plain-language answer. **Done in an afternoon.**

Value Delivered

- Domain experts contribute through **natural language** and document review
- Stakeholders get answers through **conversation**, not static reports or database queries
- AI handles both **population** (mechanical) and **analysis** (analytical)
- OSLC discovery makes this **generic** — same MCP server works with any domain
- The model becomes a **living asset** the organization both maintains and interrogates

The Big Picture

oslc4js Platform Architecture

```
Consumers     oslc-browser (humans)    oslc-mcp-server (AI)
                        ↕                          ↕
Client            oslc-client (RDF, auth, content negotiation)
                        ↕
Services      oslc-service (discovery, shapes, queries, dialogs)
                        ↕
Protocol       ldp-service (W3C Linked Data Platform)
                        ↕
Storage       storage-service interface
              ↕          ↕          ↕          ↕
         Jena/Fuseki   FS    MongoDB   ...your tool here
```

Each layer is reusable. Swap storage, add a domain, connect a new consumer — the platform stays the same.

The Pattern: Define — Instantiate — Activate

Define your domain:

1. Write a vocabulary (the terms and relationships of your domain)
2. Write resource shapes (the constraints on each type)
3. Write a catalog template (the services you want)

Instantiate your data:

4. Start the server — you have a working OSLC tool
5. Point AI at it via MCP — populate from existing documents

Activate the value:

6. Point oslc-browser at it — stakeholders browse immediately
7. Point oslc-mcp-server at it — AI queries and reasons over live data
8. Connect other tools — linked data across the enterprise

Other domains this fits: systems and software engineering, IT service management, regulatory compliance,

Three Barriers → Three Layers

Layer	Barrier	oslc4js Solution	Value
Define	Creating models is too hard	Declarative vocabularies, shapes, and catalog templates	Domain experts describe, platform builds
Instantiate	Connecting domains requires subject matter expertise across domains	OSLC linked data + discovery protocol + AI via MCP	Link by URI, discover automatically, AI creates and links
Activate	Consuming requires specialized skills	oslc-browser + MCP + SPARQL/LQE	Anyone browses; AI populates and answers; tools integrate

The Opportunity

- **oslc4js** is open source — try it with your domain
- **MRM** is a live reference implementation — see it in action today
- The barrier between domain knowledge and formal models is **dissolving**
- The question is: **which domains do you apply it to first?**

Live Demo

Creating, connecting, and consuming the Municipal Reference Model